# VERA — Remaining Work

> Extracted from `PROJECT_PHASES.md`. Items already completed are omitted. Reference the phase docs for full context, exit criteria, and design notes.

---

## Phase 1 — Hardware Bring-Up

Smoke-test scripts for each component (`scripts/test_<device>.py` exits 0).

- [x] Dual microphones — obsolete: reSpeaker now handles mic inputs and output path
- [ ] Sabrent USB audio — playback smoke test script

> DS3218 servo, NF-A6x25 fan, TMP117, stereo cameras, and Hailo-8 bring-up are functionally exercised by the running services. Formal smoke-test scripts (`test_servo.py`, `test_fan.py`, `test_tmp117.py`, `test_cameras.py`, `test_hailo.py`) may still need writing/cleanup.

---

## Phase 3 — Perception

- [x] **VAD (Voice Activity Detection)** — detect when someone is speaking
- [x] **Wake-word detection** — trigger phrase ("Hey Assistant" or similar)
- [x] **Streaming STT (Speech-to-Text)** — convert mic audio to text in real time

> Face/person detection (Hailo-8 SCRFD) and stereo depth localization are complete. Audio perception is blocked on reliable microphone input — verify mic hardware before starting.
>
> **Exit criteria:** ≤150 ms face detect latency (met); reliable wake-word at 2 m.

---

## Phase 4 — Assistant Logic

- [x] **Intent router** — classify utterance into action category
- [x] **Dialog / state manager** — track conversation context, multi-turn flow
- [x] **End-to-end voice round-trip** — wake → STT → intent → response → TTS

> TTS pipeline and Sabrent audio out are complete (Phase 2 `av_service`). This phase is blocked on Phase 3 audio perception.

---

## Phase 5 — Animation & Personality

- [ ] **Idle micro-movements** — subtle random motion when no face is detected (random motion exists but needs refinement and personality tuning)
- [ ] **Audio-reactive head motion** — head moves slightly in sync with TTS output

> Gaze tracking (pan servo follows detected face) is complete.

---

## Phase 6 — Integration & Packaging

- [ ] **Update / rollback script** — `scripts/update.sh` that pulls, bumps, restarts
- [ ] **User-facing documentation** — README how-to for end-user operation
- [ ] **Cold-boot soak test** — verify boot-to-ready < 60 s; 24 h uptime test

> systemd units (`desktop-assistant-core`, `desktop-assistant-thermal`) and config layout are complete. `/etc/desktop-assistant/` path may need review vs current `config/` layout.

---

## Future Upgrades

- [ ] **Train custom "Hey VERA" wake word** — replace the bundled `hey_jarvis_v0.1` model with a model trained on the phrase "vera" or "hey vera".

  **Why:** "Hey Jarvis" works but is an awkward activation phrase for a robot named VERA. A custom model trained on VERA's acoustic environment will also have fewer false triggers.

  **Approach (hybrid — recommended):**

  1. Record 5–10 short WAV clips of "hey vera" spoken naturally on the Pi: `da record 10` (say the phrase 2–3× per clip, vary speed and inflection).
  2. On a dev machine (GPU preferred, CPU workable), install training deps: `pip install openwakeword[train]`
  3. Train with real clips + synthetic augmentation:

     ```
     python -m openwakeword.train \
       --phrase "hey vera" \
       --positive_reference_clips ./my_recordings/ \
       --output_dir ./hey_vera_model \
       --n_samples 5000
     ```

  4. Copy the output `.onnx` to: `~/.local/lib/python3.x/site-packages/openwakeword/resources/models/hey_vera_v0.1.onnx`
  5. Update `config/assistant.yaml`: `oww_model: hey_vera_v0.1`
  6. Restart the daemon: `sudo systemctl restart desktop-assistant-core`

  **Fallback:** Synthetic-only training (omit `--positive_reference_clips`) takes ~20 min and produces a usable model without any recordings. Accuracy ~80–90%.

  **Alternative phrases also available without training:** `hey_marvin_v0.1`, `hey_mycroft_v0.1` (already bundled). Switch by changing `oww_model` in config.

- [ ] **Object detection beyond COCO-80** — every YOLO HEF in Hailo's Model Zoo (v8/v11/YOLO26, both chips) recognises the same 80 COCO classes.

  1. **Hailo-10H, no compile:** add YOLO-World v2s (Model Zoo v5.1.0 HEF) as an optional open-vocabulary mode. Second input is 80×512 CLIP text embeddings; raw outputs need host-side decode + NMS.
  2. **Compile project, both chips:** YOLOv8 trained on Open Images V7 (601 classes) via the Hailo Dataflow Compiler (x86 + NVIDIA GPU host). First prove the toolchain with a stock YOLOv8s,

then swap in OIV7 weights; verify with `scripts/hailo_smoke_test.py`.
3. **Stretch:** YOLOE prompt-free (~4.5k classes) or YOLO-World with a baked-in LVIS vocabulary; watch the large class head on Hailo-8.

- [ ] **Silent servo replacement** — swap DS3218 for Feetech STS3032 or Dynamixel XL430-W250 (half-duplex TTL UART, near-silent, position feedback built in). Requires `src/hardware/servo.py` rewrite + Pi hardware UART freed up. See `PROJECT_PHASES.md § Future Upgrades` for wiring and library details.

## Waveshare ESP32-C6 LCD

Implementation backlog for the Waveshare ESP32-C6-LCD-1.47. The LCD process runs as Arduino C++ firmware on the ESP32 and communicates with VERA through the established BLE GATT path.

- [x] **Define hardware and display requirements** — target the non-touch ESP32-C6 board with its 1.47-inch 172×320 ST7789 SPI TFT and BLE 5. The primary UX is an animated emotional mouth (neutral, listening, speaking, happy, sad, surprised, and error); the secondary UX is compact startup, restart, version, ready, degraded, and error text/iconography. Confirm exact GPIO mapping and display orientation from the board schematic during implementation. See the manufacturer documentation.
- [x] **Create the managed Arduino firmware project** — add the ESP32 Arduino source, sketch metadata, board configuration, library dependencies, and project documentation under `firmware/vera_display`.
- [x] **Establish the Arduino CLI toolchain** — use the latest suitable Arduino CLI workflow to install/configure the ESP32 board core and required libraries, with reproducible project-local build commands in `firmware/`.
- [x] **Implement the ESP32 BLE GATT display protocol** — add advertising, service/characteristic handling, framing, validation, reconnect behavior, and parsing compatible with `DisplayService`.
- [x] **Implement the expressive mouth renderer** — render the primary animated emotional mouth states with smooth timing and a safe neutral/error fallback.
- [x] **Implement startup/status rendering** — render concise boot progress, service status, ready, degraded, version, and error information as text or icons appropriate to the display resolution. Implemented in `status_renderer.h/.cpp` with color-coded states (white info, bright lime ready, yellow degraded, red error), word wrapping, and auto-reverting ready-hold timeout.
- [x] **Automate firmware build and upload** — provide Arduino CLI commands or scripts for compiling and, when the device is connected and permissions allow it, uploading firmware to the ESP32. Surface missing port, board, dependency, and permission errors explicitly. Verified end-to-end against a Waveshare ESP32-C6-LCD-1.47 connected via USB.
- [x] **Integrate host events with the ESP32 firmware** — extend the host `DisplayService` and configuration to send mouth/emotion commands and startup/status messages while preserving disabled/unconfigured behavior. Mouth/emotion path done (v1.51.0/1.52.0): `vera display mouth <state>` CLI, `display.set_mouth_state` bus topic, `DisplayService.set_mouth_state()` validation. Startup/status message path verified live end-to-end via daemon restart and bus publish events.
- [ ] **Validate the Waveshare device end to end** — test BLE discovery and reconnect, mouth animation, startup/status rendering, orientation, refresh performance, and failure recovery; document wiring, flashing, and operation. BLE discovery/connect/write, live mouth-state rendering, and

startup/status text rendering verified end-to-end against the physical device; reconnect-after-drop and long-run reliability still to validate.

- [x] **Design emotion-accurate mouth shapes** — RESOLVED in v1.57.0. `mouth_renderer.cpp`/`status_renderer.cpp` now draw into an offscreen `Arduino_Canvas` framebuffer (eliminating the flicker/redraw artifacts from direct-to-panel SPI draws) and render genuine per-state curves instead of a uniform rounded-rect pill: happy is an upward smile arc, sad a downward frown arc, speaking a pulsing open oval, surprised a round open-mouth ellipse, with neutral/listening/error remaining flat/rounded bars. Implemented with `Arduino_GFX::fillEllipse()` and a parabolic `fillCircle()`-swept stroke for the smile/frown curves. Flashed and visually validated on the physical Waveshare ESP32-C6-LCD-1.47, including a follow-up tuning pass on the speaking animation's bounce amplitude/period that was itself causing flicker at the renderer's 15fps cap.
- [ ] **Prototype silent OpenClaw personality gestures** — observe one explicitly configured owner conversation read-only and drive temporary acknowledging, thinking, satisfied, skeptical, and error gestures on the display/head without TTS or external forwarding. Keep disabled by default, avoid private/non-owner prompts, isolate OpenClaw schema integration, and validate the full prompt/completion/error sequence on real hardware.
- [ ] **Presence-aware resting mouth** — show a very small frown while no person is visible, transition to today's very small smile once a face is seen, and after the face is lost wait a configurable cooldown before easing back to a flat, neutral mouth (then the small frown). Drive from the face detection/tracking bus events; keep timings in `config/`, and make sure speaking/listening/emotion states still take priority over the resting face.
- [ ] **Video playback from the display SD card** — load video files onto the ESP32-C6's microSD card and play them on the 172×320 panel. Host-side transcoding will likely be needed (e.g. ffmpeg to MJPEG / JPEG frame sequence at panel resolution and a frame rate the C6 can decode). Define the upload path (BLE is too slow for bulk files — evaluate USB serial or Wi-Fi), plus play/stop/list commands in `DisplayService` and the CLI. Evaluate the audio side: the board has no audio output, so extract the soundtrack and play it through VERA's speakers in sync with the frames (reuse the A/V sync measurements from v1.88.0).
- [ ] **Live camera feed on the mouth display** — stream VERA's camera video to the mouth display. Downscale/crop on the Pi and send compressed frames; BLE throughput (~tens of KB/s) likely limits this to a low frame rate, so evaluate Wi-Fi streaming on the C6. Add an explicit on/off control, respect privacy mode, and return to the mouth renderer when stopped.

## Audio

- [x] reSpeaker mic capture — RESOLVED in v1.41.0 via `src/audio/pw_input.py` (`PipeWireMicInput`): a non-blocking `pw-record` subprocess feeds the AudioCaptureService. Clean, bounded shutdown (restart ~4s). Config: `audio.default.input_device_name: pipewire`.
- [x] **Web GUI volume control still non-functional** — RESOLVED (v1.61.1). Two independent faults, neither in the v1.60.0 sink-cache logic:
    1. `desktop-assistant-media.service` was still running a pre-v1.60.0 process (started before the fix landed and never restarted), so it kept executing the old `check=True` `set_volume()` that raised `CalledProcessError` on a stale sink ID instead of invalidating the cache and retrying.

2. 21 FastAPI routes were declared `async def` while performing blocking ZMQ RPCs, so every music/podcast/audio call stalled the shared event loop — making even trivial GETs take 13-23s and the UI appear dead. Fixed by converting those routes to sync `def` (FastAPI then runs them in a threadpool) and restarting the media service. Verified: slider writes land exactly (75/35/65 -> 0.75/0.35/0.65) with zero errors.

## Web UI & Home Automation

- [ ] **Re-tool home automation and add Matter devices** — restructure the web GUI home-automation section around the Matter hub (v1.86.0): list commissioned Matter devices with live state and controls, support commissioning/removal from the UI, and fold existing integrations (radon, DROP water, etc.) into a consistent device model.
- [x] **Web UI reorganization — Phase 1 (Foundation) and Phase 2 (Home)** — ES-module hash router (`#/home`, `#/media`, `#/vision`, …), Normal / Advanced / Engineering header toggle with `data-min-mode` gating, fixed card layout (drag-to-reorder retired), and a new default **Home** page. *(v1.89.0)*

Remaining web UI reorganization phases (layout per the approved plan; feature pages are for use and monitoring, Settings is for configuring; tag detail with `data-min-mode="advanced|engineering"`):

- [ ] **Phase 3 — Media page** — unified now-playing with transport; Pandora stations/thumbs; podcasts search/subscribe/episodes/seek; volume, mute, voice volume; [A] EQ preset + custom bands; [E] A/V sync panel (applied delay, display lag, output latency, spectrum fps, BLE write/ack latency, `lead_ms` knob). Backend: expose `display.spectrum_latency` + calibrator state (`GET /api/av-sync`) and `PUT` for `lead_ms` / `extra_output_ms`. Reserve a slot for video playback.
- [ ] **Phase 4 — Vision & Motion page** — Cam 1 feed + Save Frame; [A] Cam 2 and fps overlays; pan/tilt sliders + new **Center** button; Describe scene, [A] **Find object** (`/api/vision/find`); People (rename/delete, [A] merge/ guests/re-identify/train, [E] raw IDs/seen counts); [A] scene-awareness status; [E] depth (stereo/mono enable, maps, mono stats via `/api/depth/mono/stats`); [E] tracking tuning.
- [ ] **Phase 5 — Voice & Personality page** — speak text, **Tell a joke** / **Say the time** buttons (`/api/joke`, `/api/time`); skills list with enable/disable, [A] skill config; [A] record/playback; [E] STT transcripts; [E] mic analyzer + test tones. Reserve a slot for mouth-display behavior.
- [ ] **Phase 6 — Smart Home placeholder** — keep the dynamic IoT cards and add/configure modals; add a "Matter devices — coming soon" card (real Matter UI is the item above).
- [ ] **Phase 7 — Settings consolidation** — second-level left-rail entries, deep-linkable (`#/settings/<section>`): General (room name, quiet hours, greeting), Camera, Vision (object detection, room checking, privacy tuning), Cloud AI, Motion (servo, face tracking, idle motion, new **person-seek** toggle via `/api/settings/person-seek`, travel limits, [E] PCA9685 channels), Audio (mic gain, [E] backend, [E] XVF3800), Voice pipeline, Cooling ([E] fan curve/blend, [E] fan override), Updates, Power & About.
- [ ] **Phase 8 — Engineering section** (Engineering mode only) — telemetry detail, processes, services + restart daemon (confirm), event log, latency dashboard, API/CLI reference. Backend: `GET/PUT /api/fan/override` mirroring `vera fan` (keep the 100% fail-safe) and a rate-capped `/ws/bus?topics=…` live bus watch behind the API key.
- [ ] **Phase 9 — Cleanup** — retire the legacy Overview and System tabs once their content has moved, finish extracting `app.js` into page modules, remove dead code, update layout tests, docs, and the architecture diagram.

- [ ] **Idle screensaver (LCARS standby display)** — after a user-defined period with no mouse, keyboard or touch input, the web UI switches to a full-screen LCARS standby view showing the VERA logo, stardate and current time. Any mouse, keyboard or touch activity dismisses it and returns to the last page shown, with its scroll position kept. Setting: an idle timeout in Settings (with an "off" choice), saved on the device so every browser uses it. While the standby view is up, pause the camera streams to save bandwidth and CPU. Follow the LCARS design guide, use the chosen display units and time format, and respect reduced motion.

---

## Security & Privacy (Public-Deployment Hardening)

Items required before VERA could be safely exposed outside a trusted home network.

### □ Critical

- [ ] **TLS / HTTPS** — API key and camera streams travel in plaintext. Deploy an nginx reverse proxy with Let's Encrypt (or a self-signed cert for LAN) in front of the FastAPI service. All HTTP → HTTPS redirect; WSS for WebSocket streams. `?key=` in URLs will still appear in logs — move to cookie or Authorization header once TLS is in place.
- [ ] **Brute-force / rate-limit protection** — no lockout on the auth middleware. Add per-IP request rate limiting (e.g. `slowapi`) and a lockout after N failed key attempts. Log failures.
- [ ] **Replace single API key with session auth** — one shared secret means no per-user accountability and no revocation granularity. Replace with username+password login → short-lived JWT (or secure httpOnly cookie). Eliminates the localStorage XSS risk.

### □ High

- [ ] **Biometric data consent & retention policy** — face embeddings, thumbnails, and photos are stored indefinitely with no consent mechanism, retention window, or GDPR/CCPA right-to-delete workflow. Add configurable auto-expiry and a documented deletion path for bystander faces.
- [ ] **RBAC (read vs admin roles)** — all authenticated users currently have full access including hardware control (servo), face deletion, and system reboot. Define at minimum a `viewer` role (camera + face list read-only) and an `admin` role (all operations).
- [ ] **Audit logging** — no record of who accessed what, when, or from where. Log all authenticated requests (timestamp, IP, route, HTTP status) to a tamper-evident append-only log file. Essential for breach detection.
- [ ] **Telegram bot command confirmation** — destructive commands (reboot, shutdown, face-clear) issued via Telegram have no secondary confirmation. Require a confirmation reply before executing.

### □ Medium

- [ ] **OpenClaw API key storage** — `~/.openclaw/api-keys.env` is readable by any process running as `starter`. Move to a secrets manager or at minimum into the same `/etc/desktop-assistant/secrets.env` (mode 600) pattern used for VERA keys.

- [ ] **WebSocket idle timeout** — authenticated WebSocket connections have no server-side idle timeout. Add a server-side ping/pong timeout (~5 min) to close abandoned connections.
- [ ] **CORS policy** — no explicit CORS headers. Lock `Access-Control-Allow-Origin` to the specific trusted origin(s) rather than leaving it open.
- [ ] **Sudoers `kill` scope** — current whitelist allows `kill` broadly. Restrict to specific PIDs or service-restart patterns only.

## □ Low / Operational

- [ ] **API key rotation tooling** — key was generated once at install with no rotation mechanism. Add a `vera key rotate` CLI command that generates a new key, updates `secrets.env`, restarts services, and prints the new key.
- [ ] **Suppress version disclosure** — TTS boot announcement and `/health` endpoint expose the exact version string. Minor information disclosure; easy to gate behind auth or make configurable.